Assignment 2

This assignment contains 4 theoretical exercises and 5 programming exercises.

- Solutions to all theoretical exercises must be submitted in a **single** file in PDF format.
- Solutions to all programming exercises must be submitted in a **single** Java file, completing the template that you can find in the course moodle website. The file must be self-contained and must work with Java 17 or greater. You cannot use any external Java libraries. If you need them, you are allowed to use `java.util.ArrayList`, `java.util.LinkedList`, `java.util.Stack`, and `java.util.Queue`.

Theoretical exercises

Exercise 1. [10 pts]: A data structure for select and rank.
Let A be an array of n booleans. In this exercise, we write 0 instead of `false` and 1 instead of `true`. Suppose that m of these booleans are 1s and the remaining $n - m$ booleans are 0s. The `rank` and `select` operations are defined as follows:

- If $0 \leq i \leq n$ then `rank`(A, i) is the number of booleans that are true in the range $A[0], \ldots, A[i-1]$. More precisely, `rank`$(A, i) = \sum_{j=0}^{i-1} A[j]$.
- If $0 \leq j < m$ then `select(`A`, `j`)` is the position of the j-th true in the array. Note that we start indexing from 0; that is, the first true in the array is referenced by $j = 0$.

For example if A is the following array:

$$A = \begin{array}{cccccc} 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 2 & 3 & 4 & 5 \end{array}$$

then:

$$\texttt{rank(}A\texttt{, 0)} = 0 \quad \texttt{rank(}A\texttt{, 3)} = 1 \quad \texttt{rank(}A\texttt{, 4)} = 2$$

$$\texttt{select(}A\texttt{, 0)} = 0 \quad \texttt{select(}A\texttt{, 1)} = 3 \quad \texttt{select(}A\texttt{, 2)} = 4$$

a) Design a data structure with the following operations and the specified worst-case time complexities:

- **new** `RankSelect(Array<Boolean> A)`: Initializes the data structure from the given array.
  Worst-case time complexity: $O(n)$, where n is the size of the given array.
- **int** `rank(int i)`: Returns `rank(A, i)`.
  Precondition: $0 \leq$ `i` $< n$.
  Worst-case time complexity: $O(1)$.

b) Show that just by using the `rank` operation, and without storing additional information in the data structure, you can implement the following operation:

- **int** `select(int j)`: Returns `select(A, j)`.
  Precondition: $0 \leq$ `j` $< m$, where m is the number of 1s in the array `A`.
  Worst-case time complexity: $O(\log n)$.

Exercise 2. [10 pts]: Multi-way sorting.
Let A be an **array of arrays of integers**, such that A has n elements, and such that for each $0 \leq i < n$ we have that $A[i]$ is an array of m integers, sorted increasingly. We want to obtain an array of $n\,m$ elements containing all the elements, sorted increasingly.

a) Consider the following algorithm, which keeps a partial result B:

- Let $B := A[0]$ be the first array in the list.
- For each i from 1 to $n - 1$:

  Merge the partial result and the next array in the list.
  More precisely: $B :=$ Merge$(B, A[i])$.

- Return B.

Analyze the worst-case time complexity of the proposed algorithm.

b) Propose an algorithm that takes $O(n\,m\log n)$ time.

Exercise 3. [15 pts]: A data structure for anagram checking.
Two strings w_1, w_2 are **anagrams** if one is a permutation of the other. For example, "`listen`" and "`silent`" are anagrams. The following strings are pairwise anagrams: "`abc`", "`acb`", "`bac`", "`bca`", "`cab`", "`cba`". Suppose that we work over words in the latin alphabet $\{\texttt{a}, \texttt{b}, \ldots, \texttt{z}\}$. Design a data structure which represents a set of words, with the following operations and the specified worst-case time complexities:

- **new** `StringSet()`: Creates an empty set of strings.
  Worst-case time complexity: $O(1)$.
- **void** `insert(String w)`: Inserts a string w in the set.
  Worst-case time complexity: $O(m)$, where m is the length of the string w.
- **int** `countAnagrams(String w)`: Returns the number of anagrams of the string w in the set.
  Worst-case time complexity: $O(m)$, where m is the length of the string w.

For example:

```
StringSet s = new StringSet();
s.insert("abc");
s.insert("cba");
s.insert("silent");

s.countAnagrams("abc") ⇒ 2
s.countAnagrams("bca") ⇒ 2
s.countAnagrams("silent") ⇒ 1
s.countAnagrams("listen") ⇒ 1
```

Exercise 4. [15 pts]: A data structure for ranking players.
A set of n players takes part in a game. Players are identified by numbers $0, 1, \ldots, n-1$. During the game, sometimes players may earn points. Suppose that at a given moment each player i has p_i points. We would like to have a data structure that tells us the current **ranking** of a player. The ranking of a player is defined as the number of players with more (or equal) points. For example, if table of points is the following:

Player (i)	**Points** (p_i)
0	40
1	75
2	100
3	80
4	75

Then the ranking of player **2** is 1 because there is exactly one player with more or equal points. The ranking of player **3** is 2 because there are exactly two players with more or equal points. Note that many players may share the same ranking. For example, in the table above, players **1** and **4** both have ranking 4. Design a data structure with the following operations and the specified worst-case time complexities:

- `new Ranking(int n)`: Initializes the data structure with n players numbered from 0 to $n-1$. Each player starts with 0 points (so they all have the same ranking: 0).
  (No restrictions on the time complexity).
- `void score(int i)`: Gives 1 point to the player i.
  Precondition: $0 \le i < n$.
  Worst-case time complexity: $O(\log n)$.
- `int ranking(int i)`: Returns the ranking of player i.
  Precondition: $0 \le i < n$.
  Worst-case time complexity: $O(\log n)$.

Programming exercises

The goal of the following programming exercises is to implement a map (dictionary) whose keys are strings and whose values are elements of type `T`. The map will be represented using a De La Briandais tree, or "DLB tree" for short.

Recall that in a DLB tree each node has the following information:

- A boolean indicating if there is a key present in this node or not.
- A value of type `T` associated to the key.
- A character. The key of the current node extends the key of the parent node with the given character.
- A pointer to the right sibling. The siblings of the current node also represent extensions of the parent.
- A pointer to the first child. The children of a node represent extensions of the current node.

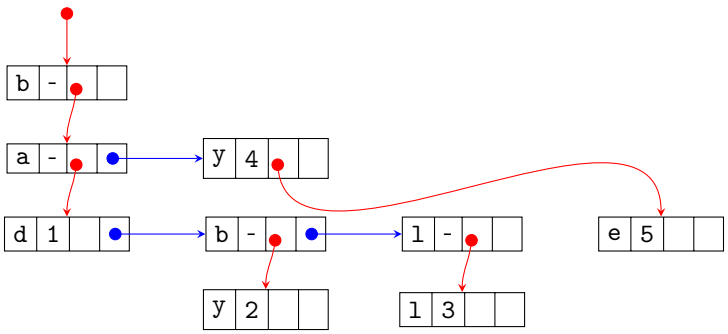
More precisely, in JAVA notation:

```
class Node {
    boolean keyPresent;
    T value;
    char symbol;
    Node sibling;  // Right sibling.
    Node child;    // First child.
}
```

The DLB tree itself is represented with a pointer to a starting node. For example, the map

$$\{\texttt{bad} = 1, \texttt{baby} = 2, \texttt{ball} = 3, \texttt{by} = 4, \texttt{bye} = 5\}$$

can be represented with the following DLB tree:



The invariant is the following:

- The structure must be acyclic, that is, there must be no loops.
- If a key is not present in the map, then the corresponding node does not store a value.
  More precisely, let `p` be a node in the structure. Then `p.keyPresent` = `false` implies `p.value` = `null`.
- A list of siblings must not include duplicated letters.
  More precisely, let `p` and `q` be different nodes in the structure such that `p` $\xrightarrow{\texttt{sibling}} \cdots \xrightarrow{\texttt{sibling}}$ `q`.
  Then `p.symbol` $\neq$ `q.symbol`.
- There should not be "useless" subtrees that do not contain any key.
  More precisely, let `p` be a node in the structure. Then `p.child` = `null` implies `p.keyPresent` = `true`.

Your task is to implement the operations below, with the specified worst-case time complexities. You can use recursive or iterative code, at your convenience. **Important:** To analyze time complexities, you can assume that the number of distinct characters (i.e. the size of the alphabet) is a constant.

Exercise 5. [10 pts]: Insertion.
Download the code skeleton and edit the `DLBMap<T>` class in the `Main.java` file.
Implement the **void** `add(String key, T value)` method, which should define a key associating it with the corresponding value. If the key is already present, this method should overwrite the value.
Precondition: the key must not be the empty string.
Worst-case time complexity: $O(m)$, where m is the length of the given key.

Exercise 6. [5 pts]: Lookup.

a) Implement the method **boolean** `contains(String key)`, which returns `true` if and only if the key is contained in the map.
Worst-case time complexity: $O(m)$, where m is the length of the key.

b) Implement the method `T get(String key)`, which returns the value associated with the given key.
Precondition: the key must be contained in the map.
Worst-case time complexity: $O(m)$, where m is the length of the key.

Exercise 7. [15 pts]: Deletion. Implement the method **void** `remove(String key)`, which removes the key from the map. If the key is not contained in the map, this method should have no effect. *Important:* Remember to preserve the last point of the invariant. More precisely, when you remove a key corresponding to a leaf, the node becomes "useless" (it has no children and no value), and it should be removed from the tree. Note that if the parent has only one child and no key, removing the useless child causes the parent to become useless, which must in turn be removed, and so on.
Worst-case time complexity: $O(m)$, where m is the length of the key.

Exercise 8. [15 pts]: Listing all the keys. Implement the method `ArrayList<String> allKeys()` which returns a list containing all the keys which are present in the given map.

Exercise 9. [5 pts] Testing
We use the following nomenclature:

- A string w_1 is a *prefix* of a string w_2 if w_2 can be written as w_1 followed by another string w'. For example, `he` is a prefix of `hello`.
- A string w is an *extension* of a string u if u is a prefix of w. For example, `hello` is an extension of `he`.
- Two strings w_1, w_2 are *disjoint* if w_1 is not a prefix nor an extension of w_2. For example, `hello` and `helicopter` are disjoint.

Complete the `DBLTest` class, adding test cases to validate that your implementation implements at least the following behavior:

a) Insert a key w_1. Check that key is in the map with the correct value. Check that none of its prefixes are in the map. Consider an extension w_2 of w_1 and check that w_2 is not in the map.

b) Repeat the previous test, but inserting two disjoint keys.

c) Insert a key w_1 and then an extension w_2. Check that keys are in the map with the correct value. Check that none of the extensions of w_1 which are prefixes of w_2 are in the map.

d) Repeat the previous test, but insert w_2 before w_1.

e) Insert all the strings of length at most 3 formed with `a` and `b` in lexicographic order (shorter strings first), with different values, and check that they have the correct values. Check that the set of keys contains all the keys.

f) Repeat the previous test but, after the first round of insertion, overwrite all the values with new values and re-check that they have the correct values.

g) Insert all the strings of length at most 3 formed with `a` and `b` in reverse lexicographic order (longer strings first), with different values, and check that they have the correct values.

h) Design test cases to validate that key removal is working correctly. Find examples that should trigger deletion of "useless" subtrees.

Submission

- You must submit your solution via the course moodle website.
- The deadline for submission is **Thursday 2026-05-28 until 23:59:59**, Shantou time (GMT+8).
- **Late submissions** will be accepted until Friday 2026-05-29 23:59:59 with **10% penalty**.
- No submissions will be accepted after Friday 2026-05-29 23:59:59.
- Sometimes the course moodle experiences delays. Please **do not wait to submit until the last minute**.